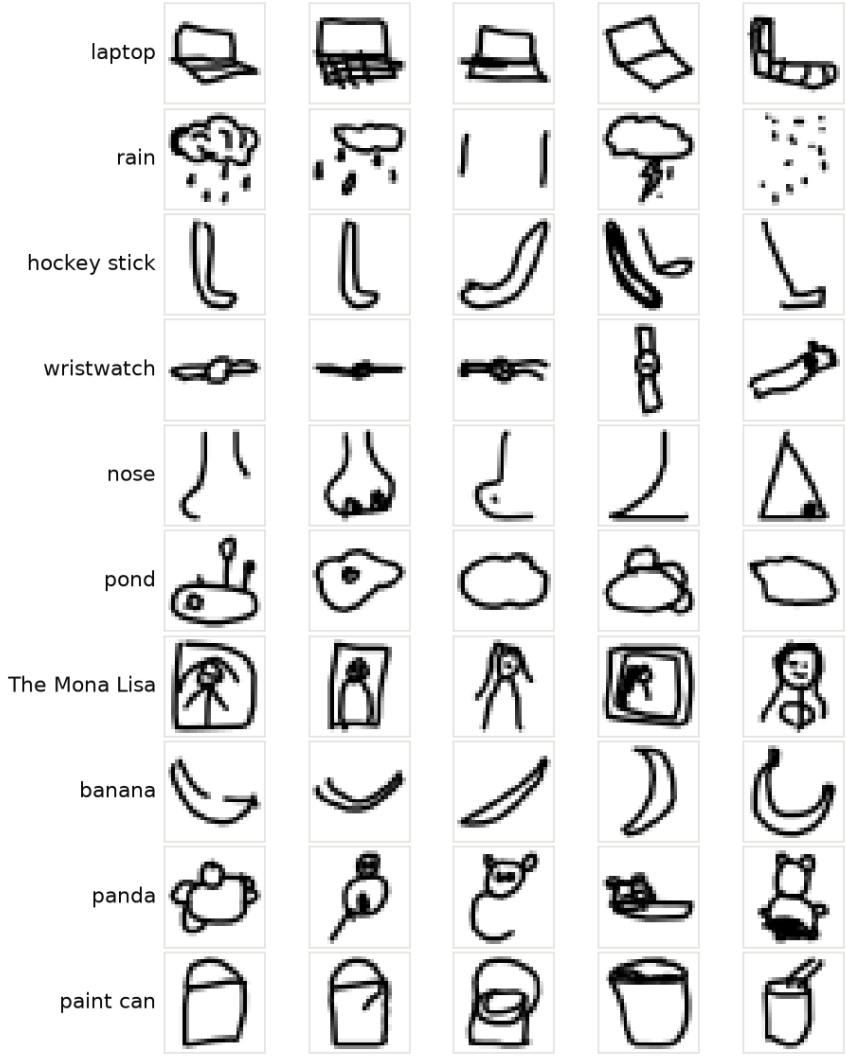
# Drawing: A New Way To Search — Reproduce & Extend

[Name 1, SRN] · [Name 2, SRN]

- Goal: classify hand-drawn doodles; the reference's motivation is drawing as a search interface
- Data: Google QuickDraw, 28×28 grayscale bitmaps
- 3 / 10 / 50 classes, 20,000 drawings per class
- Split per class 80/10/10: 16,000 / 2,000 / 2,000 (train / validation / test)



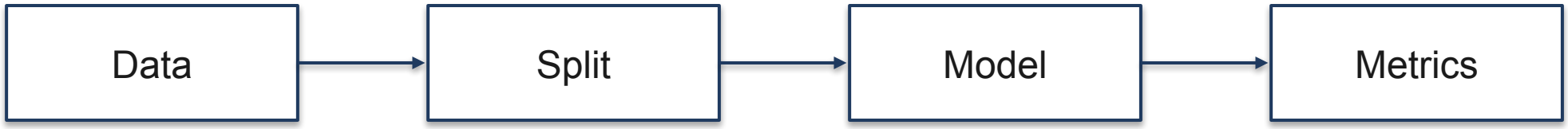
# Reference approach (the original paper)

| Family | Models | Why it was used | Key settings |
|---|---|---|---|
| Logistic Regression | LR | Simple, fast baseline | lbfgs, 100 iterations, raw pixels |
| SVM | Linear, RBF, poly (deg 5), sigmoid | Tests kernel non-linearity | 500 iterations, inputs scaled to [−1, 1], 3 and 10 classes only |
| CNN | V1–V4 | Learns spatial patterns | V1 = 2 conv + 2 max-pool + Dense 128, simplified to V2–V4; Adam, batch 500, 100 epochs; raw and binarized inputs |
| Transfer learning | VGG19, InceptionV3, ResNet50, MobileNet | Reuses ImageNet knowledge | Frozen base + small head, 3 classes |

Metrics: validation accuracy and training time (report §5.1).

# Our reproduction setup

- Same data, split, models, hyperparameters and metrics as the report
- One fixed train / validation / test split, shared by every model
- RTX 4070 Laptop 8 GB, i7-14650HX; Python 3.12.14, TensorFlow 2.21.0 (Keras 3.15.1), scikit-learn 1.9.1, XGBoost 3.4.1
- Transfer learning on features precomputed once by the frozen networks: same maths as frozen training, 30.9 min in total instead of [PLACEHOLDER: ~16 GPU-hours, not in results/]
- We also read the authors' original code to explain differences

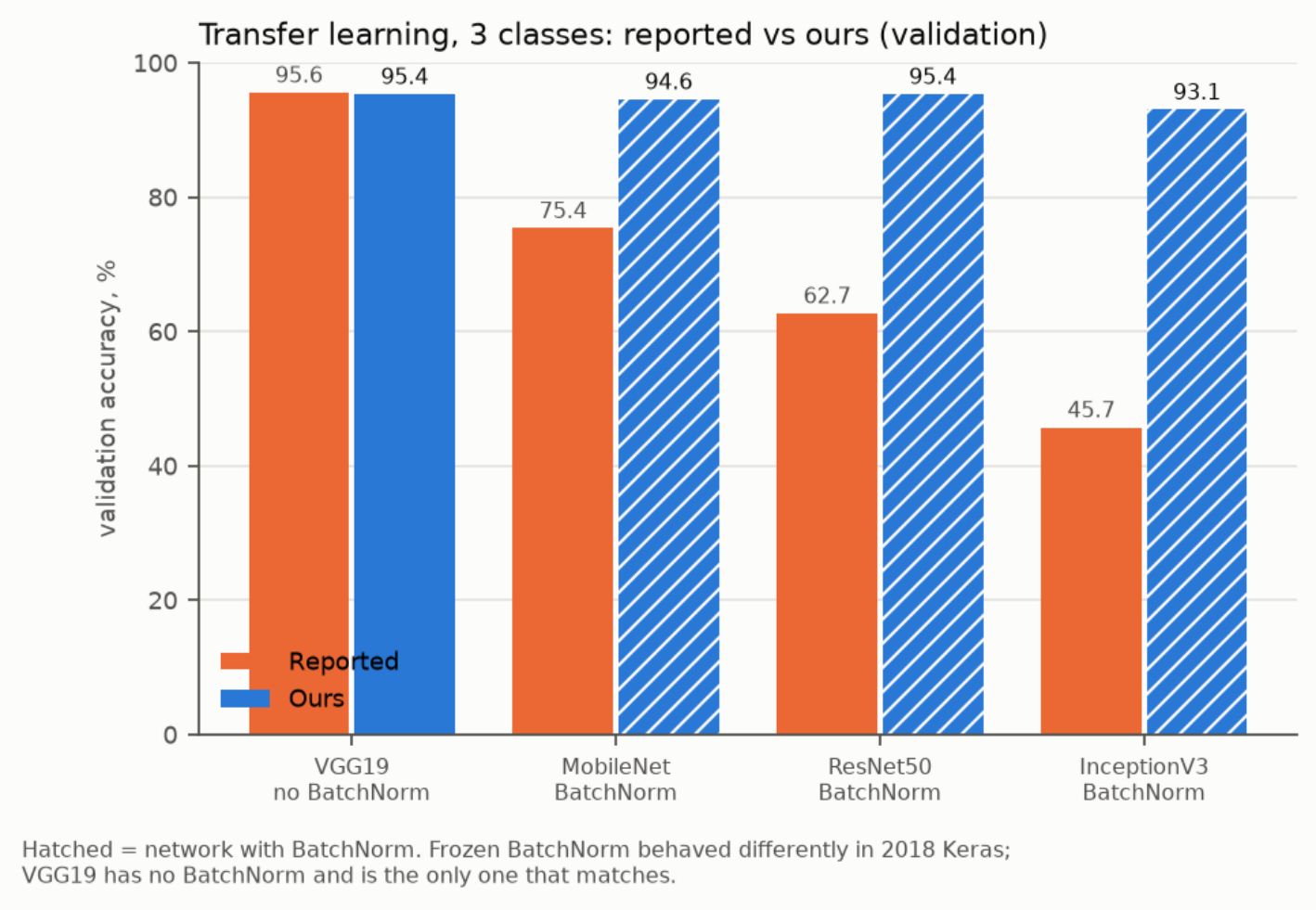


# Reproduced results: LR, SVM, CNN (validation %)

| Model | Classes | Reported | Ours | Ours, 70k |
|---|---|---|---|---|
| LR | 3 | 79.5 | 79.3 | — |
| LR | 10 | 64.6 | 65.8 | — |
| LR | 50 | 43.9 | 43.3 | — |
| SVM RBF | 3 | 62.7 | 84.0 | 75.0 |
| SVM RBF | 10 | 61.0 | 74.6 | 64.3 |
| SVM poly | 3 | 51.7 | 66.7 | 54.2 |
| SVM poly | 10 | 50.9 | 73.9 | 56.6 |
| CNN V1 raw | 50 | 81.8 | 81.2 | — |
| CNN V2 raw | 50 | 81.8 | 75.1 | — |
| CNN V3 binarized | 50 | 70.3 | 65.1 | — |

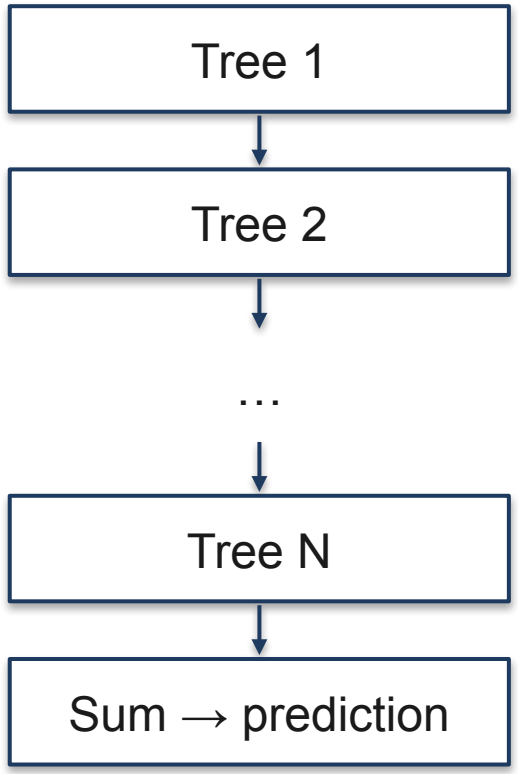
- LR within 1.2 points at all class sizes
- SVM RBF / poly above the report: its runs used ~70,000 drawings per class under the same 500-iteration cap; our 70k rerun moves them back toward it
- Our CNN V1 (81.2) matches the reported "V2" (81.8); our true V2 gets 75.1: the original code ran V1 under the V2 label
- CNN V3: 2.5 / 5.2 points below at 10 / 50 classes, unexplained

# Reproduced results: transfer learning (3 classes)

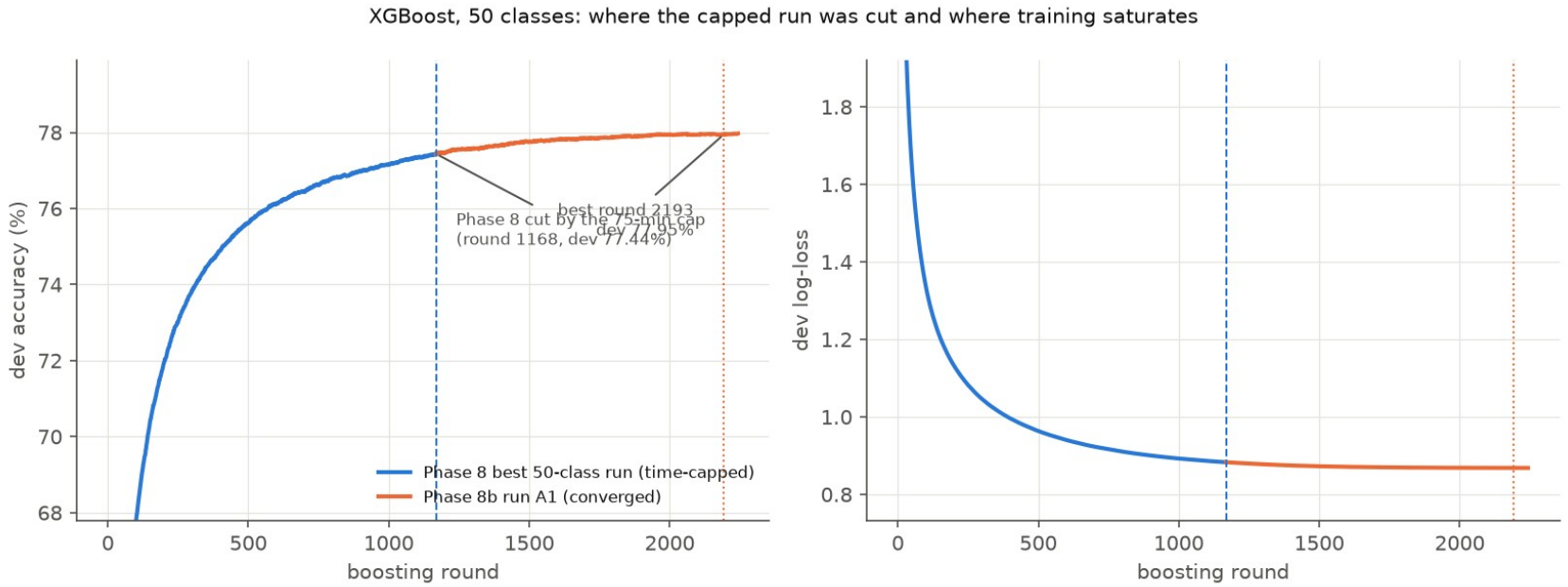



- VGG19 (no BatchNorm) matches: 95.4 vs 95.6 reported
- BatchNorm networks far higher: MobileNet 94.6 vs 75.4, ResNet50 95.4 vs 62.7, InceptionV3 93.1 vs 45.7
- Cause: frozen BatchNorm behaved differently in 2018 Keras; their Inception log shows [PLACEHOLDER: ~100% training, not in results/] vs 45.7% validation
- So "complex networks don't help doodles" is likely an artefact

# Extension: XGBoost (gradient-boosted trees)

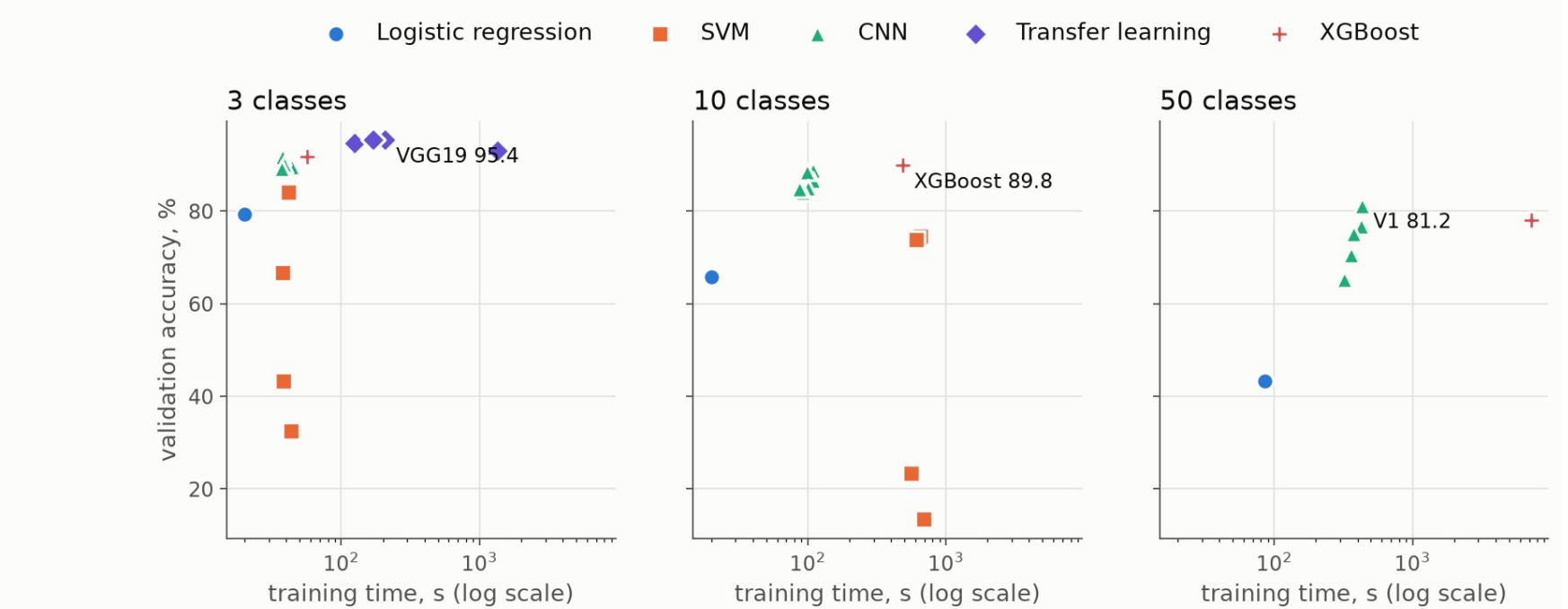



Each tree corrects the errors of the trees before it




- Why: the reference had no tree-based model, so a fourth model family
- Optuna (TPE, Bayesian) on validation: 45 / 31 + 32 trials at 3 / 10 classes; top 4 settings on 50
- Phase 8 time-capped; 8B uncapped, wider search. Test % at 3 / 10 / 50: 90.8 / 89.7 / 77.4 → 90.8 / 89.8 / 77.9
- Every 50-class setting plateaued at about 78%

# Comparison of all models (our runs, validation accuracy)




| Classes | LR | SVM (RBF) | CNN (best) | TL (VGG19) | XGBoost |
|---|---|---|---|---|---|
| 3 | 79.3% · 20 s | 84.0% · 41 s | 91.7% · 38 s | 95.4% · 207 s | 91.7% · 57 s |
| 10 | 65.8% · 20 s | 74.6% · 655 s | 88.8% · 109 s | — | 89.8% · 486 s |
| 50 | 43.3% · 86 s | — | 81.2% · 431 s | — | 78.0% · 7,191 s |

# Key findings

- CNN is best on 50 classes: 81.2% in 431 s, about 16.7× faster than tuned XGBoost (78.0%, 7,191 s)
- XGBoost matches CNNs on 3 and 10 classes (91.7 vs 91.7; 89.8 vs 88.8) but plateaus at about 78% on 50: trees judge single pixels, CNNs learn shapes
- Three inconsistencies in the reference explained: the CNN version bug, the SVM data size, the frozen BatchNorm effect
- Future work: feed transfer-learning features into XGBoost or LR, as the report suggests